Systems Literacy

Module 01

The Anatomy of a Machine

Course Description

This curriculum transitions learners from casual computer use to foundational systems understanding, preparing them for Operating Systems, Networks, Architecture, and Programming.

Course Outcomes

By the end of this 5-lecture series, learners will be able to:

- Clearly distinguish the roles and interactions of hardware, operating systems, and application software within a complete computing system.
- Understand how information is represented in binary, encoded, stored as files, and interpreted by software.
- Grasp how CPUs, memory, storage, and buses coordinate to execute programs and move data efficiently.
- Interpret system performance signals to diagnose bottlenecks (CPU, RAM, disk, I/O) and build a mental model sufficient to progress into OS, networks, and programming.

Agenda

- The Hardware Trinity
- The Boot Process
- Modern I/O
- The Abstraction Stack
- Hands-on Lab

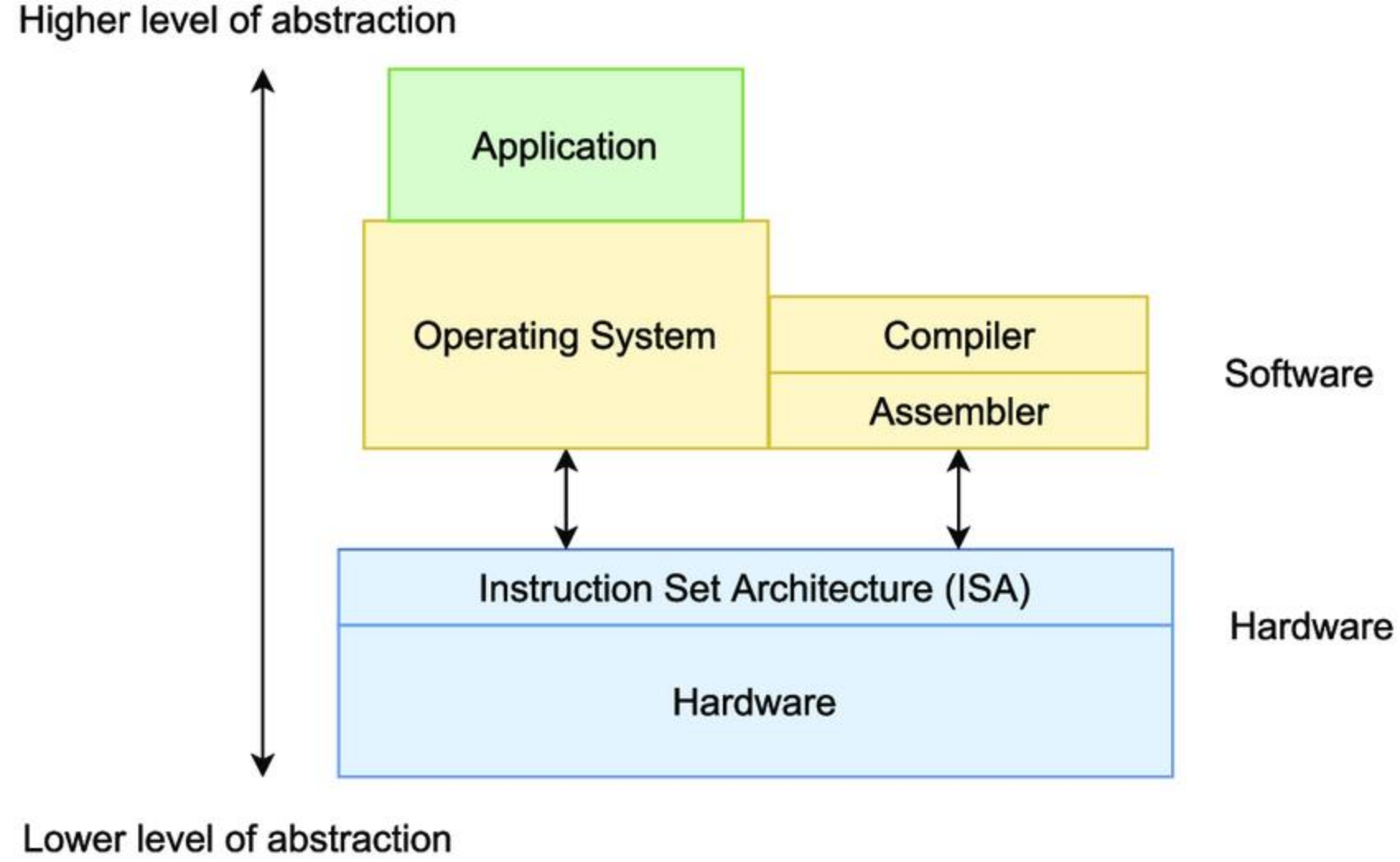
The Computing Stack

Computers are built on layers of **Abstraction**. Each layer hides the complexity of the layer below.

- The Physical Layer: The actual silicon, copper traces, and electrons.
- The Firmware/Kernel Layer: The "translator" that allows software to speak to hardware.
- The User Layer: Where applications (browsers, games) live.

As a systems-literate user, you need to know which layer a problem is occurring in. A "slow" computer could be a hardware failure (Physical) or a background update (User Space).

Layers of Abstraction



The Hardware Trinity

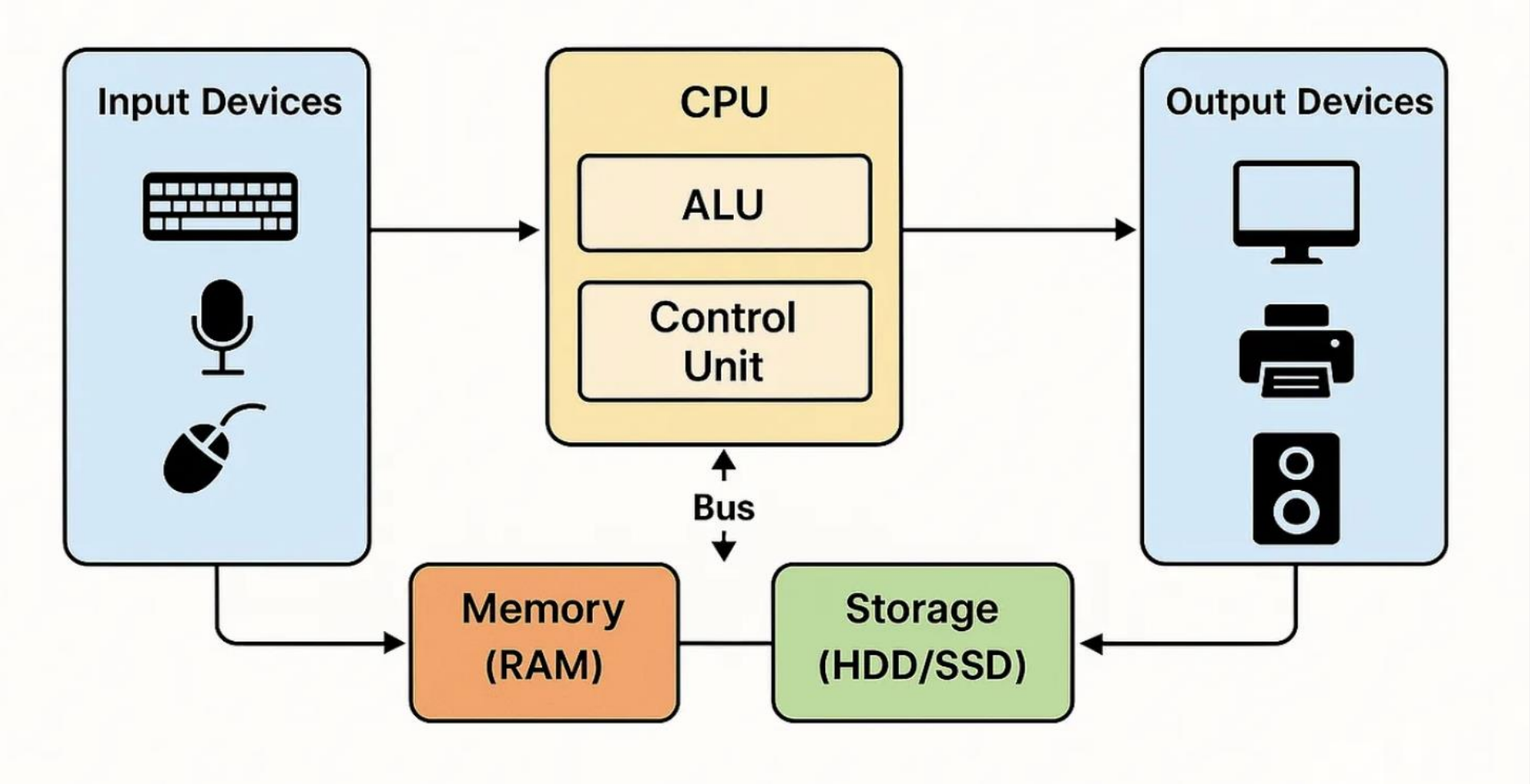
Every computer, from a calculator to a supercomputer, relies on the interplay of three core components:

1. The **Central Processing Unit**: The "brain" of a computer. It executes instructions using the fetch-decode-execute cycle.
   - Clock Speed: Number of cycles per second (Hz) at which CPU executes instructions, determining how fast it can process operations.
   - Cores: The number of processors in a CPU.

2. **Random Access Memory**: The "short-term" memory. It is volatile (loses data without power). The RAM acts as a staging area. The CPU cannot run a program directly from the hard-drive – it must be loaded into the RAM first.

3. **Storage** (Long-Term Memory): It is non-volatile form of storage. Current standard is a NAND-Flash drive also known as Solid State Drives (SSD), evolving from the earlier standard of a Hard Disk Drive (HDD).
   - This was the single biggest performance jump in the last decade.

The Hardware Trinity



Input Output (I/O) Devices

Input Devices: How we talk to the Computer

Input devices helps us send data to the computer:

- Keyboard
- Mouse
- Microphone
- Scanner
- Camera

Output Devices: How the computers talk to us.

After processing data, computers use output devices to show the results to us. These include:

- Monitor (Screen)
- Speakers
- Printer

The Central Processing Unit (CPU)

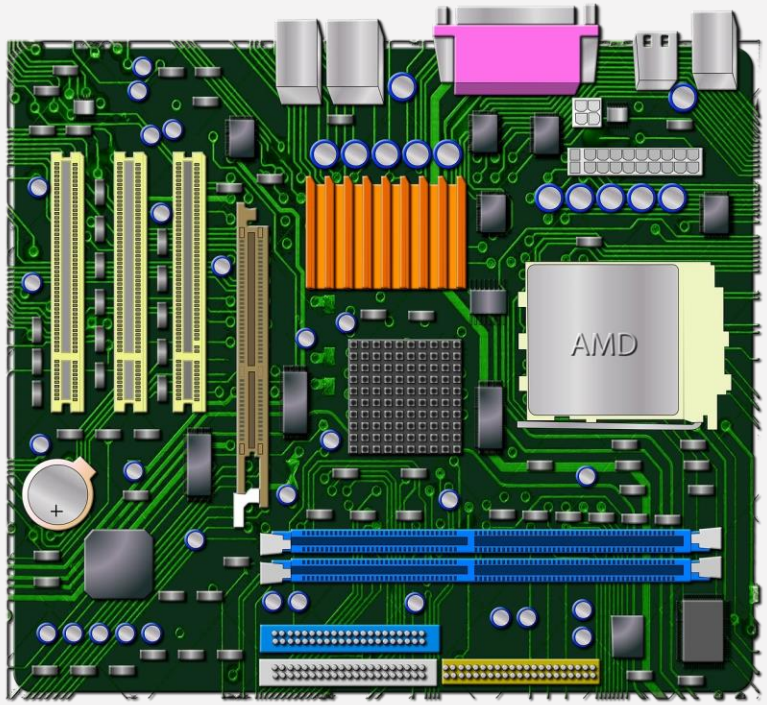
The CPU is the brain of the computer. It is the part of a computer that does the thinking. It has two key parts:

1. The Arithmetic and Logical Unit (ALU): It does the calculations and logic decisions
2. The Control Unit: It sends the signals to control everything else.
3. Registers: Very fast storage that stores information required to execute instructions.

The CPU works super-fast and needs help from the memory to do its job. It can execute billions of instructions per second.
Eg: A CPU with 3GHz clock speed and an average IPC (instructions per cycle) of 2 can execute about 6 billion instructions per second.

64-bit architectures vs 32-bit architectures: The registers in the CPU are 64 bit wide for a 64-bit architecture, while they are 32 bit wide for a 32-bit architecture.

CPU & Motherboard



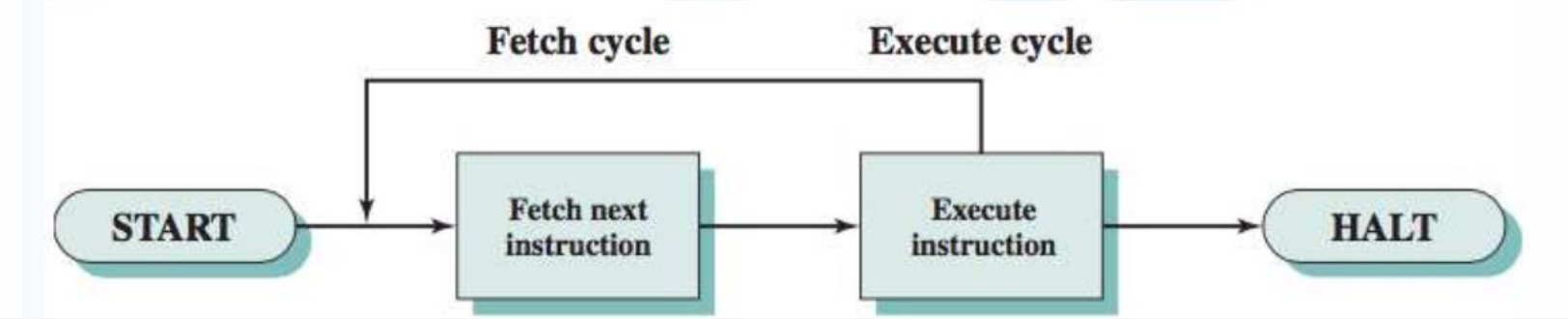
A Motherboard is the main circuit board that connects the CPU, Memory, Storage, and Peripherals, allowing them to communicate and receive power.

The Architecture of a Modern Computer

Computers work in a simple cycle: Input > Process > Output or Fetch > Decode > Execute

"This model of a computer was developed by John Von Neumann"

Fetch: The CPU reads the next instructions from the memory using the address in the Program Counter.
Decode: The control unit interprets the instruction—what operation to perform and which data or registers to use.
Execute: The CPU performs the operation (e.g., arithmetic, memory access, or a jump) and stores the result.

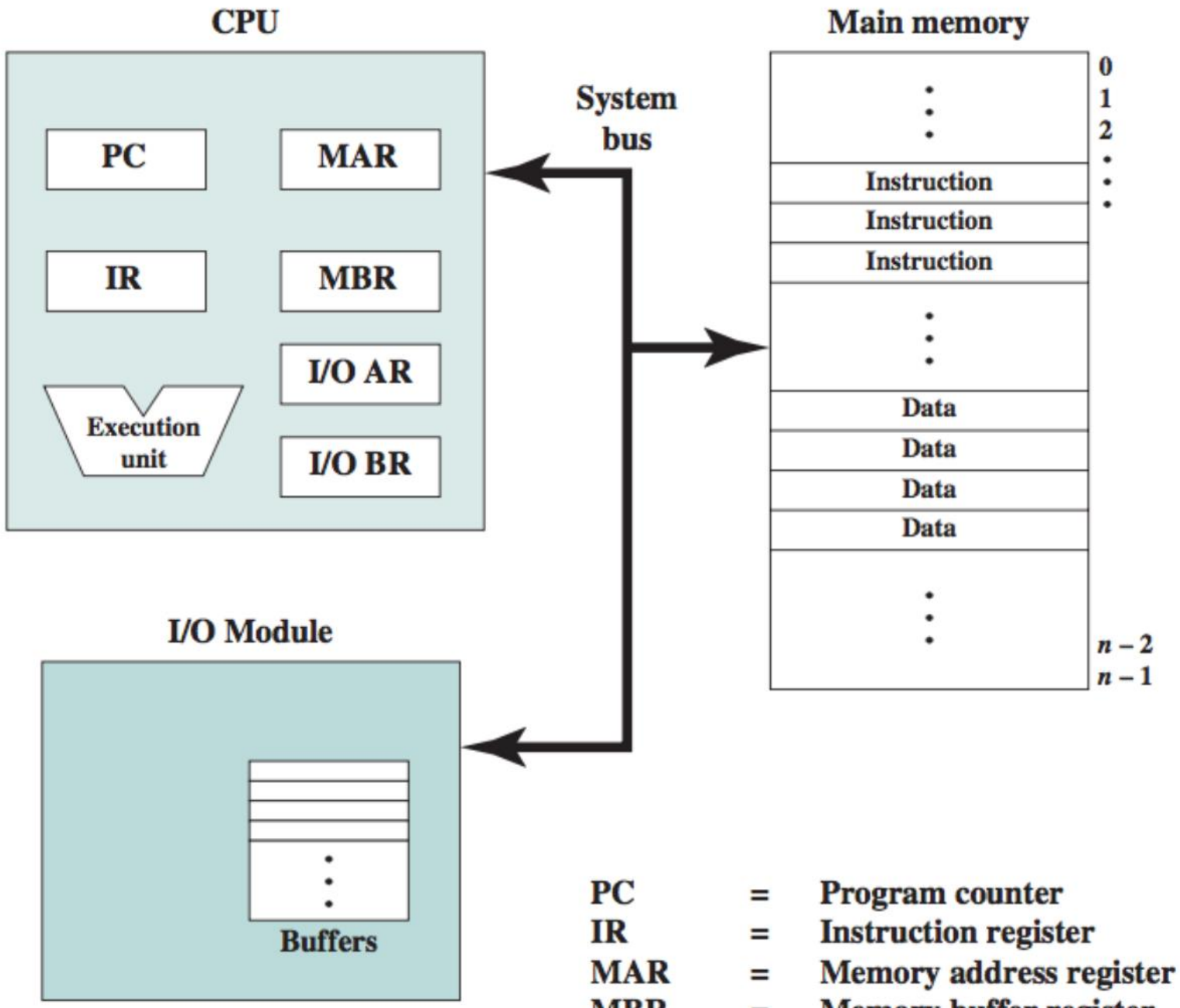


The Central Processing Unit (CPU)

The CPU fetches and executes instructions stored in the memory using internal registers like:

1. **Program Counter (PC)**: Keeps track of what instruction comes next. It stores the address of the next instruction in the main memory.
2. **Instruction Register (IR)**: Holds the current instruction.
3. **Accumulator (AC) and Multiplier Quotient (MQ)**: Hold temporary data used in calculations.
4. **Input/Output Registers**: Input/output registers temporarily hold data being transferred between the CPU and external devices, letting the CPU communicate with peripherals without directly handling slow hardware signals.

Why: The CPU needs registers because they provide **ultra-fast storage inside the CPU** to hold instructions, data, and addresses, avoiding slow memory access and enabling efficient execution of operations.



PC = Program counter
IR = Instruction register
MAR = Memory address register
MBR = Memory buffer register
I/O AR = Input/output address register
I/O BR = Input/output buffer register

Memory

The computer has two types of memory: **Short-Term** Memory and **Long-Term** Memory.

Random-Access Memory (RAM): This is a fast, short-term memory. It loses its stored data when you turn off the computer.

Storage (Hard-Drive or SSD): This is long-term memory. It keeps your files, app, and data even when the computer is off.

RAM helps the CPU work faster, while storage helps save everything you need.

Question: Where is the operating system and user applications stored?

Memory

When a computer is off, both the operating system and the programs are stored in the non-volatile storage – memory that does not need power to retain data.

Firmware (special case): A small but critical part of system software (UEFI/BIOS) is stored in a flash memory chip on the motherboard, and not on the SSD/HDD.

What happens when you turn the computer on

- **Firmware (UEFI/BIOS)** runs first from motherboard flash.
- It locates the OS on the **SSD/HDD**.
- The OS kernel and essential components are **loaded into RAM**.
- When you launch a program, its code is also **copied from storage into RAM** and executed by the CPU.

Firmware is **software that behaves like hardware**: tightly coupled, minimal, deterministic, and foundational.

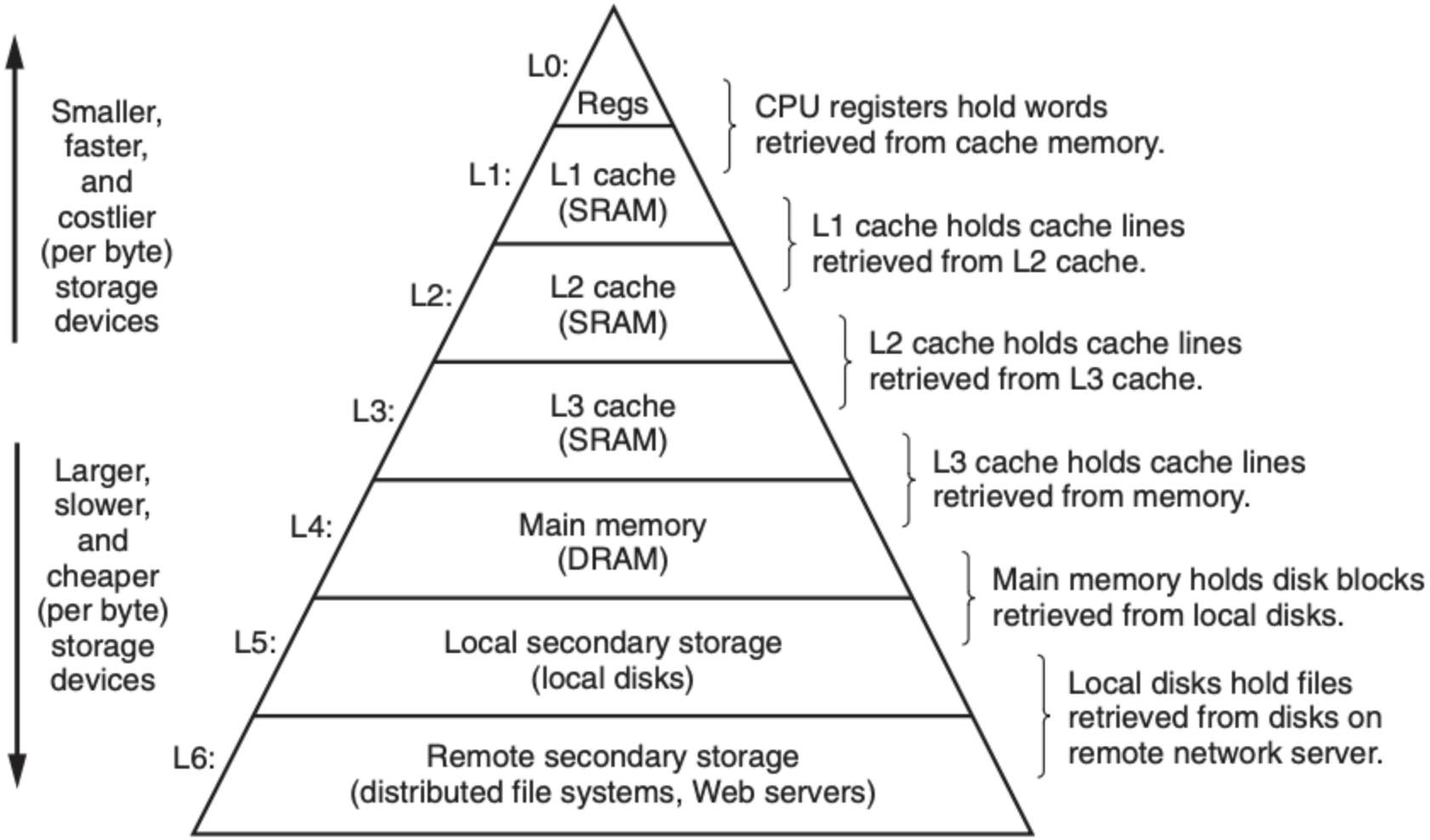


Figure 1.9 **An example of a memory hierarchy.**

The Boot Process

How a "cold" piece of silicon becomes a functional environment.

- **Step 1: POST (Power-On Self-Test):** The firmware checks if the RAM and CPU are working. If you hear "beeps" and nothing happens, the POST failed.
- **Step 2: UEFI/BIOS:** The **Unified Extensible Firmware Interface**. This is the first software that runs. It identifies the boot drive which means finding a device that contains valid boot code and deciding to hand control to it.
- **Step 3: The Bootloader:** A tiny program (like GRUB or Windows Boot Manager) that tells the UEFI/BIOS where the Operating System's **Kernel** is located.
- **Step 4: Kernel Initialization:** The OS takes control of the hardware from the BIOS.

UEFI firmware is a tiny OS that can read a FAT32 partition, and the ESP is just the place on disk where it looks for bootloader files.

Boot Process

What makes firmware different

- **Lives on the device itself** (motherboard, SSD, GPU, router, etc.)
- **Runs before any OS exists**
- **Persists without power**
- **Rarely changed**, but can be updated ("flashed")

What firmware actually does (PC example) - On a typical computer:

- **UEFI/BIOS firmware** powers on the CPU
- Initializes hardware (RAM, CPU, keyboard, storage)
- Performs basic hardware checks (POST)
- Finds a bootable device
- Loads the OS bootloader into RAM and hands over control

After that, the OS takes over.

Boot Process

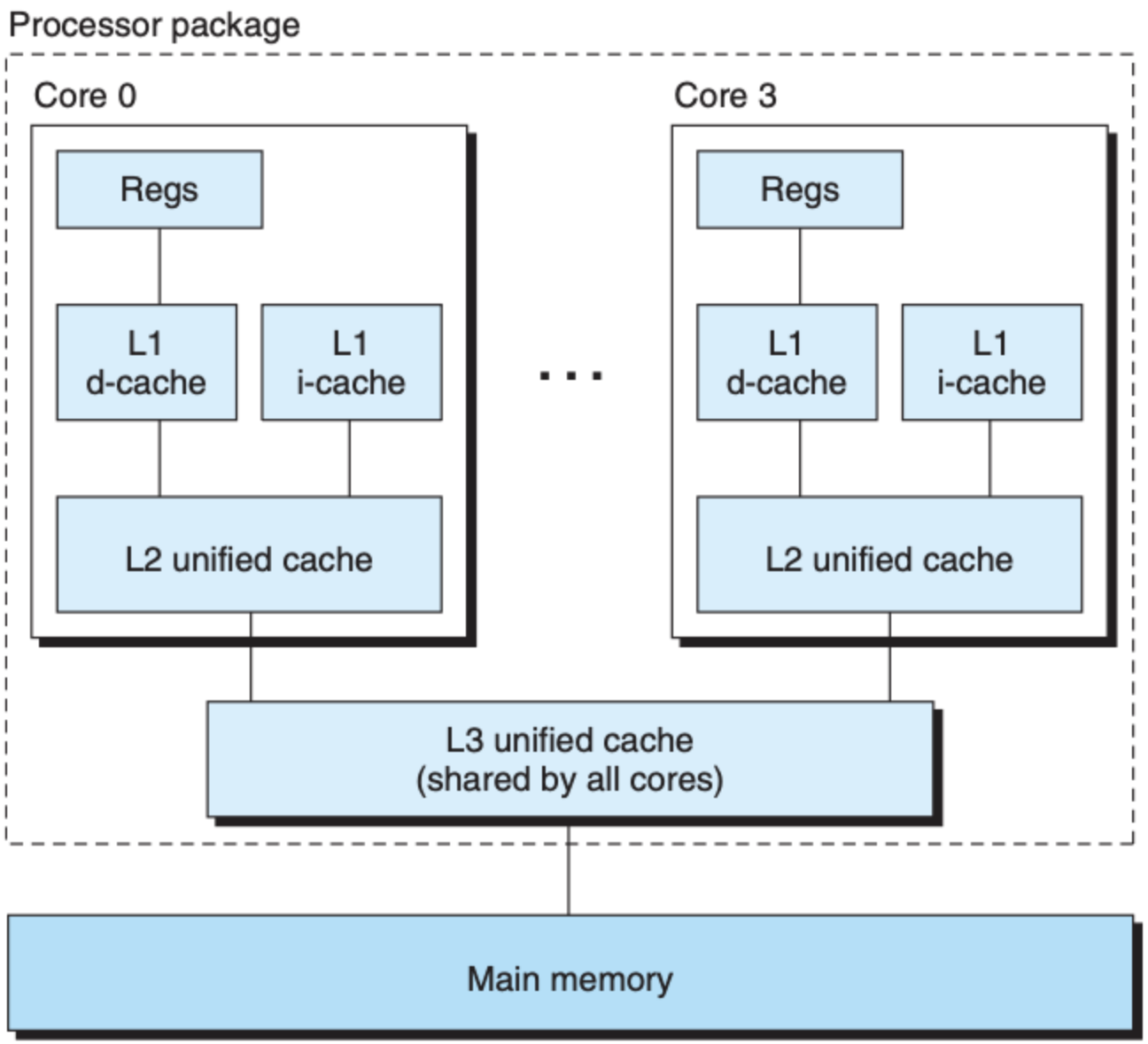
Examples of firmware:

- **BIOS / UEFI** → motherboard startup logic
- **SSD firmware** → wear leveling, error correction, caching
- **GPU firmware (vBIOS)** → graphics initialization
- **Router firmware** → networking logic
- **Keyboard / mouse firmware** → key scanning, USB behavior

Firmware vs OS vs applications (clean mental model)

Layer	Role	Runs when
Firmware	Wake up & prepare hardware	Power-on
Operating System	Manage hardware & processes	After boot
Applications	Do user tasks	After OS

Figure 1.17
Intel Core i7 organization. Four processor cores are integrated onto a single chip.



The Boot Process

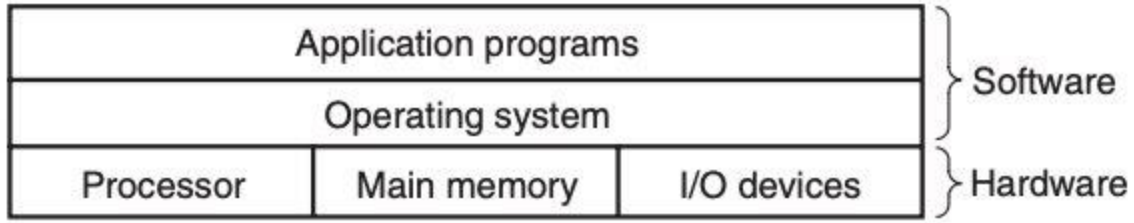
Firmware: Looks for bootable structures. The firmware does not load the OS itself. It finds something that can.
- UEFI looks for EFI System Partition (ESP)
- Legacy BIOS looks for Master Boot Record (MBR)

Bootloader: The **bootloader** exists only to get the kernel running. Once kernel starts, the bootloader is discarded.

Kernel: Performs core operations of the OS.

Operating System: Makes the computer user friendly.

Power on
↓
Firmware (UEFI/BIOS) ← stored on motherboard
↓
Bootloader ← stored on SSD/HDD
↓
Kernel ← stored on SSD/HDD, loaded into RAM
↓
Operating system



```
Disk (SSD)
├── ESP (FAT32, ~100–500 MB)
│   └── EFI/
│       ├── Microsoft/Boot/bootmgfw.efi
│       ├── ubuntu/grubx64.efi
│       └── Boot/bootx64.efi
├── OS partition
└── Data
```

High Level OS Anatomy

1. Kernel is the core of the Operating System. It
   - Manages CPU scheduling.
   - Controls Memory (RAM)
   - Talks to hardware via drivers.
   - Enforces security and isolation.
   - Provides system calls to applications.
2. Init is the first program the kernel starts.
3. Background processes that provide core OS functionality.
4. System Libraries: Provide standard APIs to programs. Applications rarely talk to kernel directly. They talk to libraries, which talk to the kernel.

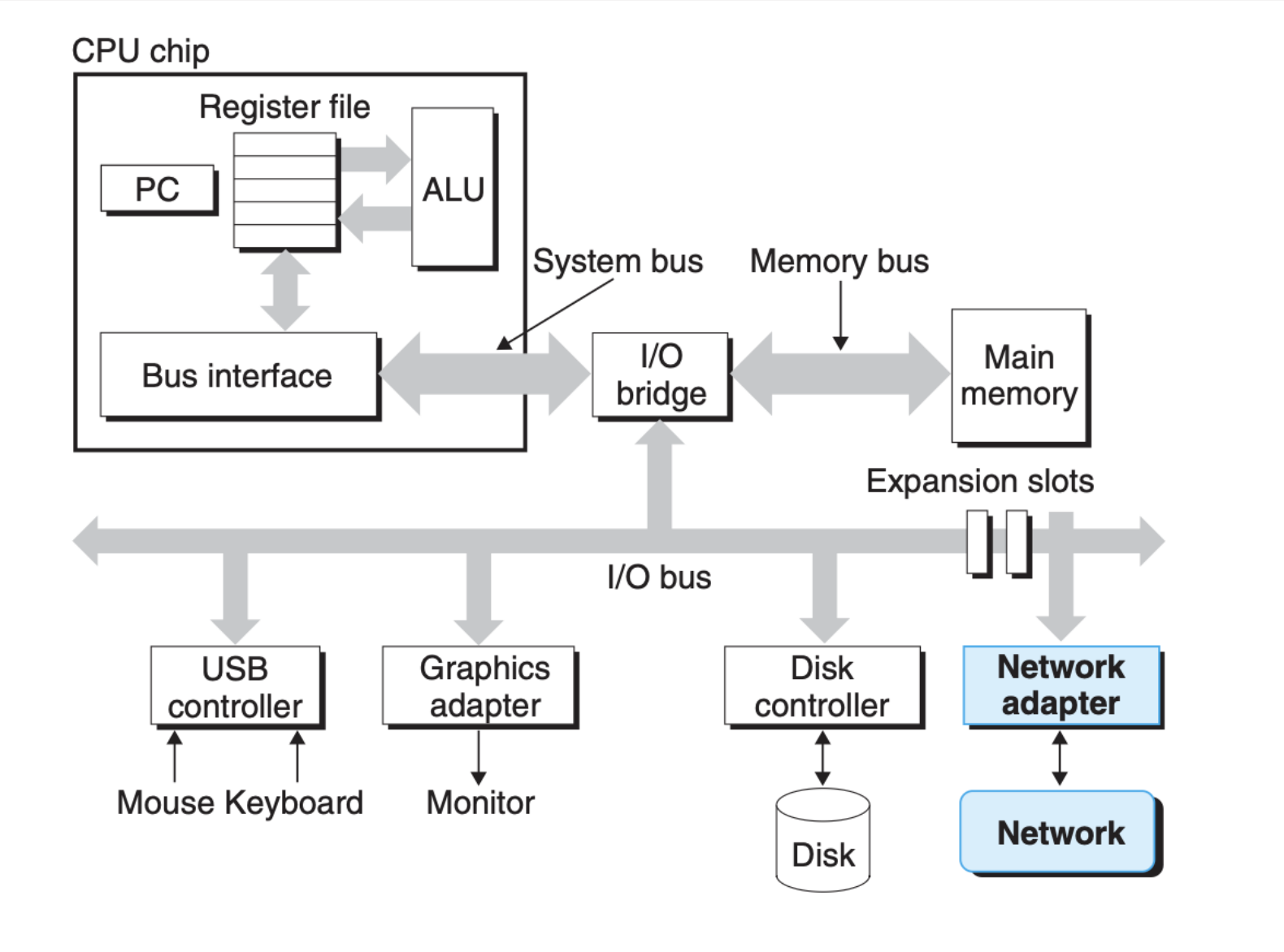
The kernel keeps the machine alive and safe; everything else in the OS exists to make it usable. Everything above the kernel runs in **user space**.

```
┌──────────────────────────────┐
│ Applications (user programs) │
├──────────────────────────────┤
│ System libraries & runtimes  │
├──────────────────────────────┤
│ System services / daemons    │
├──────────────────────────────┤
│ Init system (PID 1)          │
├──────────────────────────────┤
│ Kernel                       │
└──────────────────────────────┘
```

Modern I/O Protocols

In this section, we will learn:

- **The Bus:** Set of wires that moves data between the CPU, memory, and other parts.
- **USB-C & Thunderbolt Ports**
- **PCIe (Peripheral Component Interconnect Express):** The internal high-speed lanes used by graphics cards and NVMe drives.
- **Latency vs. Throughput:** *Throughput* is how many lanes the road has (how much data); *Latency* is the speed limit (how long it takes one bit to get there).


CPU chip
Register file
PC
ALU
Bus interface
System bus
Memory bus
I/O bridge
Main memory
Expansion slots
I/O bus
USB controller
Graphics adapter
Disk controller
Network adapter
Mouse Keyboard
Monitor
Disk
Network

Bus: Computer's Nervous System

A bus is not just a wire; it is a shared communication pathway. Think of it as a **highway system** that connects different "cities" (the CPU, RAM, and Storage).

In the past, buses were "parallel" (sending many bits at once over many wires). Modern buses are often "serial" (sending bits very fast one after another), which sounds slower but actually prevents signals from getting mixed up at high speeds.

Types of Buses in a Modern System:

- **Address Bus:** Tells the system *where* the data needs to go.
- **Data Bus:** Carries the *actual content*.
- **Control Bus:** Coordinates the "traffic lights" (read/write commands).

USB-C and Thunderbolt: The Universal Ports

These are the external "gates" to the bus. Students often get confused between the **connector** (the shape) and the **protocol** (the language/speed).

USB-C (The Shape): It's just the physical plug. It's reversible and convenient, but the "brains" behind it vary.

Thunderbolt (The Performance): Developed by Intel and Apple, Thunderbolt uses the USB-C shape but adds a massive "supercharger."

- **The "Tunnel" Analogy:** Thunderbolt is unique because it "tunnels" different types of data (DisplayPort for monitors and PCIe for data) through one cable simultaneously.
- **Daisy Chaining:** Thunderbolt allows you to plug a monitor into a drive, and the drive into the laptop, all in one line.

PCIe: The Internal Autobahn

The Concept: PCIe (Peripheral Component Interconnect Express) is the high-speed internal connection used for the most demanding tasks.

- **Lanes ($x4, x8, x16$):** Explain that PCIe uses "lanes."
- A **GPU** (Graphics Card) usually needs **16 lanes ($x16$)** because it handles massive amounts of visual data.
- An **NVMe SSD** usually needs **4 lanes ($x4$)**.
- **Generations:** Each generation (Gen 4, Gen 5, Gen 6) effectively doubles the speed of the previous one. It’s like upgrading the speed limit on the highway without adding more lanes.

Latency vs Throughout

The most critical distinction for a CS student to understand.

Term	Analogy	Technical Definition
Throughput (Bandwidth)	**The Width of the Pipe.** How much water can flow through at once?	The total volume of data transferred in a set time (e.g., 10 Gbps).
Latency	**The Length of the Pipe.** How long does it take for the first drop to come out the other end?	The time delay between a command and the start of the data transfer (measured in milliseconds or nanoseconds).

You can have high throughput but high latency (e.g., sending a box of 100 hard drives via a cargo ship). The "bandwidth" is massive, but the "latency" is days!

Lab Work (30 mins): The Inspection

- **Task A: System Profiling.** * *Windows:* msinfo32 or Task Manager > Performance.
  - *Action:* Students identify their CPU model, RAM speed (in MHz), and whether their storage is NVMe or SATA.

- **Task B: Identifying Bottlenecks.** * Open a browser with 20 tabs. Watch the RAM usage.
  - **Swap File**: What happens when RAM fills up? The OS uses the slow Hard Drive as "fake RAM." This is why computers "crawl" when memory is full.

- **Task C: Device Manager/System Report.**
  - Find a "Human Interface Device" (Mouse/Keyboard).
  - Explain **Drivers**: The small piece of software that tells the OS how to interpret the specific signals from that hardware.

Appendix

Non-Volatile Memory Express (NVMe)

NVM: Non-Volatile Memory refers to memory types that do not need power to store data.

NVMe: Non-Volatile Memory Express. The express refers to the fact that it travels over the PCI express bus (PCIe).

Before NVMe, SSDs used the **SATA** interface. SATA was originally designed for old-fashioned spinning hard drives. It was like putting a Ferrari (a fast SSD) on a dirt road (SATA) behind a slow tractor (the old SATA controller). NVMe removes that tractor. It allows the SSD to talk **directly to the CPU** via those high-speed PCIe lanes.

RAM vs. Storage: RAM is your "desk surface" (where you work right now). Storage is your "filing cabinet" (where things go when you're done). You can work much faster on a big desk than by running to the cabinet for every single paper.